

STRAW: Stress-Aware WL-Based Read Disturbance Management for High-Density NAND Flash Memory

Myoungjun Chun

mj.chun@ssu.ac.kr
Soongsil University
Seoul, Republic of Korea

Jisung Park

jisung.park@postech.ac.kr
POSTECH
Pohang, Republic of Korea

Jaeyong Lee

jylee@davinci.snu.ac.kr
Seoul National University
Seoul, Republic of Korea

Myungsuk Kim

ms.kim@knu.ac.kr
Kyungpook National University
Daegu, Republic of Korea

Inhyuk Choi

ihchoi@davinci.snu.ac.kr
Seoul National University
Seoul, Republic of Korea

Jihong Kim

jihong@davinci.snu.ac.kr
Seoul National University
Seoul, Republic of Korea

Abstract

While NAND flash memory has continuously increased its storage density over decades, this progress has exacerbated the read-disturbance problem. In this work, we identify two fundamental limitations of existing read-disturbance management techniques that trigger read reclaim (RR) at the block granularity: (i) they overlook the heterogeneous reliability impact of read disturbance across individual wordlines (WLs), leading to unnecessary RR in many cases; and (ii) they address read disturbance only after disturbance-induced errors have already accumulated, which forces substantial RR-induced copy overheads in read-disturbance-prone modern NAND flash memory. To address these limitations, we propose STRAW (STress-Aware Wordline-based read-disturbance management), a new technique that minimizes RR overheads through two key ideas: (i) stress-aware WL-based read reclaim, which monitors the accumulated read-disturbance effect on each WL and reclaims only heavily disturbed WLs, and (ii) stress-reduced read, which mitigates disturbance on valid WLs during each read operation by scaling pass-through voltages based on WL validity. Our experimental results using a modern SSD emulator show that STRAW reduces RR-induced page-copy overhead by 88.6% on average compared with the state-of-the-art technique.

CCS Concepts: • Hardware → External storage.

Keywords: solid-state drives (SSDs), NAND flash memory, read disturbance, read reclaim, SSD lifetime, I/O performance

ACM Reference Format:

Myoungjun Chun, Jaeyong Lee, Inhyuk Choi, Jisung Park, Myungsuk Kim, and Jihong Kim. 2026. STRAW: Stress-Aware WL-Based Read Disturbance Management for High-Density NAND Flash

Memory. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3779212.3790228>

1 Introduction

High-density 3D NAND flash memory is the predominant technology for modern storage systems to meet both the high-performance and large-capacity requirements of data-intensive applications. Manufacturers are consistently competing to increase the storage density of NAND flash memory via two key technologies. First, stackable 3D NAND technology enables vertical scaling, achieving hundreds of wordline (WL) layers, e.g., 321 layers [34]. Second, advanced multi-level cell technologies (e.g., triple-level cell (TLC) [59] and quad-level cell (QLC) [33]) effectively increase a NAND flash chip's storage density by allowing each flash cell to store multiple bits (e.g., three and four bits, respectively).

Such remarkable improvements in storage density, however, come at the cost of significant reliability degradation in modern NAND flash memory. A flash cell stores bit data as a function of its threshold voltage (V_{th}) level, e.g., a single-level cell (SLC) can be read as '1' (or '0') if its V_{th} level is lower (higher) than a reference voltage (V_{ref}). There exist various error sources in NAND flash memory, such as read disturbance [9, 25], program interference [6, 11, 38, 57], and retention loss [10, 12, 13, 47], which affect the V_{th} levels of programmed cells and can potentially cause bit errors. In high-density NAND flash memory, the reliability impact of these error sources significantly increases for two reasons. First, a cell's threshold voltage changes more easily as the cell size and cell-to-cell distance decrease, which is a consequence of increasing the number of stacked WL layers [2, 60]. Second, the advanced MLC technologies narrow the V_{th} margins, making even small V_{th} shifts more likely to cause raw bit errors when reading stored data [4, 5, 7, 8].

Among various error sources, read disturbance has gained increasing attention recently as a major reliability concern in modern and future high-density NAND flash memory.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790228>

Read disturbance is a phenomenon in NAND flash memory where reading a page affects the reliability of other pages in the same block.¹ To read a page, a NAND flash chip applies not only V_{ref} to the target wordline (WL) but also a pass-through voltage ($V_{pass} > V_{ref}$) to *all* other non-target WLs in the block. Doing so slightly programs the non-target WLs, which can potentially corrupt the stored data in the WLs when repeated. As the block size of high-density 3D NAND flash memory rapidly increases with continuous vertical WL stacking, reading a page affects a larger amount of data. For example, in a recent NAND flash chip with 321 WL layers [34], the amount of data stored in non-target WLs in the block exceeds a hundred MB.

Despite the increasing concern, how to effectively manage read disturbance in modern NAND flash memory has not been thoroughly investigated yet; to our knowledge, only a few prior studies have proposed read-disturbance management techniques based on *read reclaim* (RR) [25, 26, 35, 43, 45, 68, 81, 83]. As a high-level description, RR is an internal SSD management task triggered when a block's read count RC (i.e., the number of page reads performed to the block) exceeds a threshold RC_{MAX} . Once triggered, RR copies all the valid pages in the target block to free pages, which *resets* the read-disturbance effect accumulated on the block (i.e., it eliminates the transient errors in the pages caused by read disturbance). It is necessary to meticulously set RC_{MAX} to cover the worst-case operating conditions (e.g., for pages or blocks that are much more susceptible to read disturbance than others) while minimizing the copy overheads that can significantly degrade I/O performance and SSD lifetime.

Unfortunately, in this work, we find that the existing RR approach is hard to use for modern SSDs due to its two key limitations. First, the existing approach manages read disturbance at a *block granularity*, which frequently incurs *premature* RR in modern SSDs. In high-density 3D NAND flash memory, the reliability impacts of read disturbance on individual WLs differ significantly due to (i) higher disturbance impact on adjacent WLs of the target WL [61, 80] and (ii) large reliability variation among different WLs [28, 67]. This means that data loss can occur at a much lower RC under the worst-case access pattern, i.e., when only a certain page neighboring the weakest WLs in the block is read intensively, compared to when the block's pages are uniformly accessed. The probability of encountering the worst-case pattern can be very low in practice but is *not* zero, so the existing block-level approach must conservatively set RC_{MAX} low enough to guarantee data reliability under the worst-case scenarios, which causes unnecessary RR in many cases.

Second, the existing approach is *reactive*; it acts only after read-disturbance effects have accumulated sufficiently

within a block, rather than mitigating them proactively. While this approach incurs only a tolerable RR cost in conventional NAND flash memory, disturbance resistance has sharply deteriorated in modern high-density NAND flash memory. Our characterization of five real 3D NAND flash chip types shows that data loss can occur at a 88.2% lower RC in 3D QLC NAND flash memory than in 2D TLC NAND flash memory. Thus, even without premature RR invocations, modern SSDs that primarily use RR to manage read disturbance must reclaim substantially more data than conventional SSDs.

Our goal in this work is to improve the lifetime and performance of modern SSDs by reducing RR invocations. To this end, we propose *STRAW* (*STress-Aware WL-based read disturbance management*), a novel read-disturbance management technique that achieves this goal through **two key ideas**: (i) *stress-aware WL-based Read Reclaim* (WR^2), which reclaims only stressed WLs—i.e., heavily disturbed WLs—rather than reclaiming the entire block; and (ii) *Stress-Reduced Read* (SR^2), which proactively reduces read-disturbance stress by adaptively lowering V_{pass} on non-target WLs during each read.

WR^2 identifies heavily disturbed WLs in each block and reclaims them in a timely manner, minimizing premature RR. To this end, WR^2 employs a read-disturbance model—derived from an extensive characterization of 160 real 3D NAND flash chips—that quantifies (i) heterogeneous per-WL read-disturbance tolerance and (ii) the difference in per-read disturbance impact between adjacent and non-adjacent WLs. By leveraging the model with per-WL counters, WR^2 accurately estimates the impact of a page read on the reliability of each non-target WL in the same block. For an efficient implementation, WR^2 leverages the Space-Saving algorithm [49] to mitigate the space overhead for per-WL counters.

SR^2 proactively mitigates read-disturbance-induced reliability impact during read operations via an intelligent V_{pass} -scaling mechanism. As read-disturbance stress on non-target WLs decreases exponentially with V_{pass} , lowering V_{pass} is highly effective; in our characterization, a 5% reduction improves ERC_{MAX} by up to 59%. However, in high-density NAND flash memory, a naïve V_{pass} reduction can substantially increase the raw bit error rate (RBER) of the target page; for instance, even a 5% reduction can double the RBER (see §5.3). To safely reduce V_{pass} , SR^2 leverages a new finding from our characterization: increasing V_{pass} on non-target WLs lowers the target page's RBER. SR^2 adopts a two-step V_{pass} -scaling policy: (i) increase V_{pass} for *invalid* non-target WLs to ensure sufficient margin, and (ii) leverage this margin to lower V_{pass} on *valid* non-target WLs.

STRAW provides high lifetime and performance benefits with only small changes to the NAND flash chip's control logic and to the existing SSD firmware. Our real-device characterization and system-level evaluation using a state-of-the-art SSD emulator [40] show that STRAW reduces read-disturbance-induced writes and 99th-percentile read-tail

¹NAND flash memory reads and writes data in a page granularity (e.g., 16 KB), while erasing data in a block granularity (e.g., 2,784 pages [53]). See §2.1 for more details.

latency by an average of 88.6% and 66.3%, respectively, over an SSD with the state-of-the-art RR technique [81].

2 Background

In this section, we review the basics of NAND flash memory, as well as the mechanism and impact of read disturbance in modern NAND flash memory.

2.1 Basics of NAND Flash Memory

3D NAND Flash Organization. Figure 1 shows the schematic organization of 3D TLC NAND flash memory. 3D NAND flash memory boosts storage density by stacking cells along the z-axis. A *wordline* (WL) is the control-gate line shared by all cells at the same vertical position in a block. A vertical array of cells (e.g., 232 layers [53]) forms a NAND *string* that connects to a *bitline* (BL). Multiple strings across distinct BLs—typically 8K–16K—form a *block*. Thousands of blocks constitute a *plane*; multiple planes (e.g., 2 or 4) constitute a *die*; multiple dies constitute a *package* (chip).

Data Storage Mechanism. A flash cell is the fundamental building block of NAND flash memory: a unique type of transistor that modulates its threshold voltage (V_{th}) by injecting/ejecting electrons into/from the charge-trap layer. The number of electrons in the charge-trap layer determines the cell's threshold voltage (V_{th}), which encodes the stored data. In single-level cell (SLC) NAND flash memory, for example, a high V_{th} encodes '0' and a low V_{th} encodes '1'. To increase density, a cell can store multiple bits by controlling V_{th} more precisely—a technique known as *multi-level cell* (MLC) technology. A cell storing m bits (i.e., m different pages) uses 2^m distinct V_{th} states, with $m = 2$ (MLC), $m = 3$ (TLC), and $m = 4$ (QLC). As m increases, the fixed V_{th} window—set at design time—must accommodate more states, squeezing them closer together and narrowing the voltage margins.

Basic Operations. NAND flash memory supports three basic operations: *program*, *erase*, and *read*. A *program* operation increases a target cell's V_{th} by applying a high program voltage (e.g., > 20 V) to its WL, injecting electrons into the charge trap layer. An *erase* operation resets the cell's data value to '1' by applying a high voltage (e.g., > 20 V) to the substrate. This removes electrons from the charge trap layer, thereby lowering V_{th} for all cells in the target block. A *read* operation determines a cell's V_{th} level (i.e., the stored data) by applying a read-reference voltage V_{ref} to the cell's control gate, which

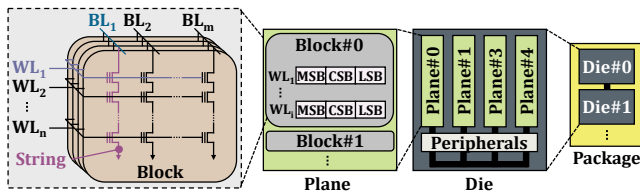


Figure 1. Organization of 3D TLC NAND flash memory.

makes the cell operate as either an on-switch (when $V_{ref} < V_{th}$) or an off-switch (when $V_{ref} > V_{th}$) depending on its V_{th} level. As a number of flash cells (e.g., more than 2^{17}) share a WL as described in Figure 1 (i.e., the same voltage is applied to every cell sharing the WL), NAND flash memory performs read and program operations at a *page* granularity.

2.2 Read Disturbance Phenomenon

Mechanism & Impact. In NAND flash memory, read disturbance is the phenomenon in which reading a target WL inadvertently disturbs non-target WLs in the same block. Figure 2 illustrates this mechanism. As shown in Figure 2(a), during a read operation, the flash chip applies a pass-through voltage V_{pass} (e.g., > 6 V) to *all* non-target WLs so that their cells act as pass transistors [9]. While V_{pass} is lower than the program voltage, it slightly programs cells on non-target WLs and shifts their V_{th} distribution rightward. When read operations are repeatedly performed on a block, the accumulated V_{th} shift leads to bit errors, as shown in Figure 2(b). If the total number of bit errors exceeds the ECC engine's correction capability, data loss may occur [18, 19, 23, 44, 77].

Unfortunately, the reliability impact of read disturbance is far more severe in high-density 3D NAND flash memory for two key reasons. First, as block sizes increase with continuous vertical WL stacking, each read operation disturbs a larger amount of data within the same block. For example, in recent 3D TLC NAND flash memory with 321 vertical layers, the block size reaches 120 MB [34], compared with 5 MB in 2D MLC NAND flash memory [42]. Since a read operation disturbs all other WLs in the same block, this leads to about 24× more data disturbed per read operation.

Second, in high-density NAND flash memory, a single read operation incurs more read-disturbance stress on non-target WLs: higher V_{pass} and longer read latency lead to larger per-read V_{th} shifts. By the Fowler–Nordheim (FN) tunneling relation [24], the disturbance effect scales with both V_{pass} and read latency. As more WLs are stacked to increase capacity, the string lengthens, requiring a higher V_{pass} to fully activate the cells in the longer string. Read latency also increases with the number of bits per cell—for example, SLC NAND flash memory reads in about 20 μ s [56], whereas QLC NAND flash memory requires about 90 μ s [16].

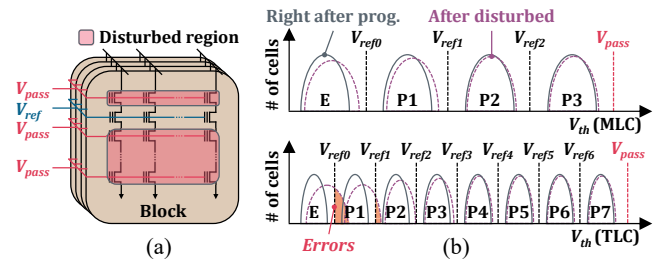


Figure 2. Mechanism of read disturbance phenomenon.

Reliability Management: Read Reclaim. To our knowledge, all prior read-disturbance management techniques—in both the literature [25, 26, 35, 43, 45, 68, 81, 83] and industrial practice (e.g., Micron [51])—are based on *block-level* read reclaim (RR). When a block’s read count RC (i.e., the number of page reads to the block) exceeds a predefined threshold RC_{MAX} , the SSD controller triggers an RR procedure to eliminate read-disturbance-induced errors by rewriting (copying) all valid pages in the block to other free pages. Flash manufacturers carefully set RC_{MAX} through extensive profiling of flash chips to prevent read-disturbance-induced data corruption, even under worst-case scenarios.

3 Motivation

In this section, we (i) evaluate the impact of RR on off-the-shelf SSDs across different generations, and (ii) discuss the limitations of existing techniques.

3.1 Negative Impact of RR on Modern SSDs

To understand the impact of RR on SSD lifetime and performance, we characterize three SSDs with varying MLC technologies and numbers of WL layers, from multiple vendors. We run Fio [3] on each fresh-state SSD for 12 hours, issuing 4 KB random reads across the entire LBA range with a queue depth of 32. During the test, we collect (i) SSD-internal write volume (i.e., RR-induced writes) by logging attribute 245 (SSD wear-out attribute) in the S.M.A.R.T. dataset [52, 62]; and (ii) read-latency samples at 4 ms intervals.²

Figure 3 compares cumulative SSD-internal write volume over 30 TB of host reads and read-latency traces for each SSD. We make two key observations. First, as SSD density increases, RR-induced write volume has become large enough to noticeably shorten device lifetime. For example, a 176-layer 3D QLC SSD incurs 8.7 TB of internal writes after 30 TB of host reads (Figure 3(a)). Assuming a 200 TBW [63], a read-only workload at 50 MB/s could wear out the QLC SSD in 144 days—far shorter than a typical three-year warranty. Second, RR-induced operations can impact user-perceived read latency in high-density SSDs; on the 72-layer TLC and 176-layer QLC SSDs, we observe significant read-latency

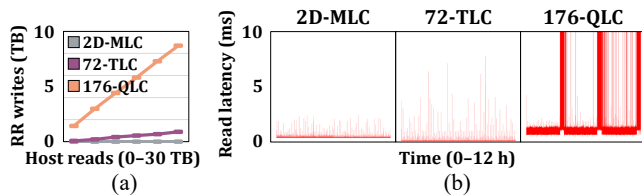


Figure 3. Impact of RR on three off-the-shelf SSDs.

²Although logging every read is ideal, it incurs enough overhead to interfere with the measured read latency. Therefore, we use a 4 ms sampling interval, which provides sufficient samples while keeping overhead negligible.

fluctuations (up to 10 ms) during repeated reads, unlike the 2D MLC SSD (Figure 3(b)). From these observations, we conclude that modern—and future—high-density SSDs require an efficient read-disturbance management technique.

3.2 Limitations of the Existing Solutions

To mitigate the negative impact of RR in modern SSDs, several prior works [25, 26, 35, 43, 45, 68, 81, 83] have proposed various RR optimizations. While these techniques have demonstrated notable improvements, we find that they still face two fundamental limitations.

Limitation 1: Block-Level Granularity. Managing the read disturbance problem at *block granularity* is highly inefficient in modern NAND flash memory. Unlike in planar (2D) NAND flash memory, where a page read disturbs all non-target WLs in the block almost equally [25], the reliability impact of read disturbance significantly varies across WLs in high-density 3D NAND flash memory. Figure 4 illustrates two key factors that contribute to the read-disturbance variations. First, when reading a page, a high-density 3D flash chip applies a higher V_{pass} (V_{passH} , $\approx 10\%$ higher than V_{pass} [75]) to the two adjacent WLs compared to the non-adjacent WLs (Figure 4(a)). As explained in §2.2, the impact of read disturbance is exponentially proportional to V_{pass} [25]. Consequently, reading a page leads to a greater impact on reliability of data stored in adjacent WLs [61, 75]. Second, read-disturbance tolerance varies significantly among WLs due to inherent process variations in high-density 3D NAND flash memory [47, 55, 66, 67, 73, 78, 79]. Figure 4(b) shows the maximum tolerable read counts per WL—i.e., the maximum number of reads that do not incur uncorrectable errors—within a block when the block’s pages are accessed uniformly (see §5.1 for a detailed description of our infrastructure and methodology). As shown in Figure 4(b), the worst WL in a block can tolerate only 403K reads before data corruption, while the best WL can reliably endure 559K additional reads.

To quantify the asymmetry in read disturbance within a high-density 3D block, we measure the raw bit error rate (RBER) of WLs in a block after repeated read operations on a real TLC flash chip. Figure 5 illustrates the change in RBER for 48 representative WLs in a block under two distinct read patterns, PA and PB, where PA reads only the page at WL₃₅,

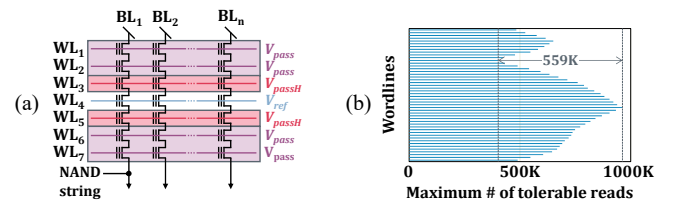


Figure 4. Key factors for asymmetry in read disturbance across WLs.

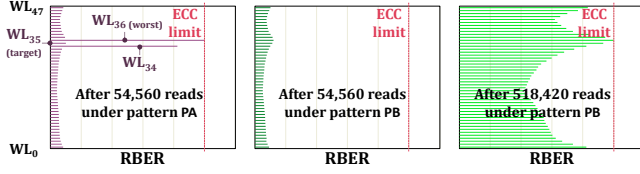


Figure 5. Heterogeneous disturbance impact under different read patterns.

which is adjacent to the worst WL (WL_{36}) while PB reads all pages in the block sequentially. As expected, the maximum read count that a block can endure without data loss varies significantly based on the read pattern. Under the pattern PA, reading WL_{35} leads to a significant increase in the RBER of its adjacent WLs (WL_{34} and WL_{36}), resulting in uncorrectable errors (i.e., RBER exceeding the ECC correction capability) after only 54,560 reads. In contrast, the block can tolerate up to 518,420 reliable reads under the pattern PB.

Due to this asymmetry, the existing block-level RR approach requires setting a significantly low RC_{MAX} to prevent data loss under the worst-case scenario (i.e., all read requests targeting a single WL), which rarely occurs in real-world workloads. For instance, in an SSD employing the flash chips shown in Figure 5, the block-level RR approach must set RC_{MAX} to 54,560. Our observations clearly indicate that the block-level RR approach is highly inefficient in modern SSDs employing high-density NAND flash memory, as it frequently triggers premature RR operations.

Limitation 2: Reactive Manner. Existing solutions for modern SSDs [45, 68, 81, 83] manage read disturbance *reactively*: they wait until disturbance-induced reliability degradation exceeds RC_{MAX} , and then trigger RR. Under this reactive policy, the overhead of RR scales inversely with the read-disturbance tolerance of NAND flash memory. Unfortunately, as explained in §2.2, high-density NAND flash memory is expected to exhibit lower disturbance tolerance because each read operation introduces more read-disturbance stress on non-target WLs [61].

To quantify this trend, we characterize five real flash chip types—2D MLC/TLC and 3D MLC/TLC/QLC. For each chip type, we measure the read-disturbance tolerance of WLs under two patterns—PA and PB—across 160 chips. Figure 6 reports, for each chip type, the *average* maximum tolerable

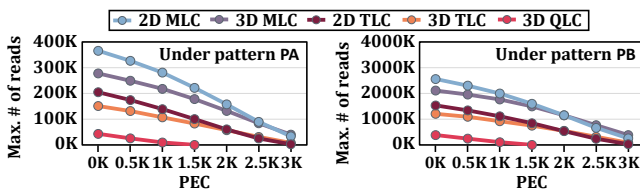


Figure 6. A comparison of the maximum tolerable read counts across different types of flash chips.

count across the evaluated WLs at different P/E cycle counts (PECs). As expected, we observe that modern high-density 3D NAND flash chips are substantially more vulnerable to read disturbance. For example, at 1K PEC under the PA pattern, the maximum read count for 3D QLC chips is 88.2% and 79% lower than for 2D MLC and 2D TLC chips, respectively. We conclude that, in high-density NAND flash memory, even an ideal RR approach—reclaiming only the necessary WLs (i.e., without premature RR invocations)—must reclaim significantly more data than in conventional counterparts.

4 STRAW: Key Mechanisms

Our goal is to design a read-disturbance management technique that (i) minimizes premature RR while ensuring data reliability, and (ii) proactively improves the read-disturbance tolerance of modern NAND flash memory to reduce the number of heavily disturbed WLs. To this end, we propose STRAW, a STress-Aware WL-based read-disturbance management technique. This section introduces the two key mechanisms behind STRAW: (i) stress-aware WL-based Read Reclaim (WR^2) and (ii) Stress-Reduced Read (SR^2).

4.1 Stress-Aware WL-based Read Reclaim (WR^2)

Key Idea. WR^2 is based on our key insight that the read disturbance should be managed at a finer granularity, i.e., per WL, not per block. Unlike the block-level RR approach, which triggers a block-level RR operation based on RC_{MAX} , WR^2 monitors the heterogeneous reliability impacts of read disturbance on individual WLs and reclaims only the heavily disturbed WLs in a timely manner.

Figure 7 depicts an example of how WR^2 manages the read disturbance at the WL granularity under two distinct read patterns, PA and PB, as shown in Figure 5. The PA pattern, which reads only the page at WL_{35} , causes a significant disturbance in the two adjacent WLs, WL_{34} and WL_{36} . Under this access pattern, as shown in Figure 7(a), WR^2 selectively reclaims the adjacent WLs, WL_{34} and WL_{36} , on time after 54,570 and 66,270 RC, respectively, allowing an additional

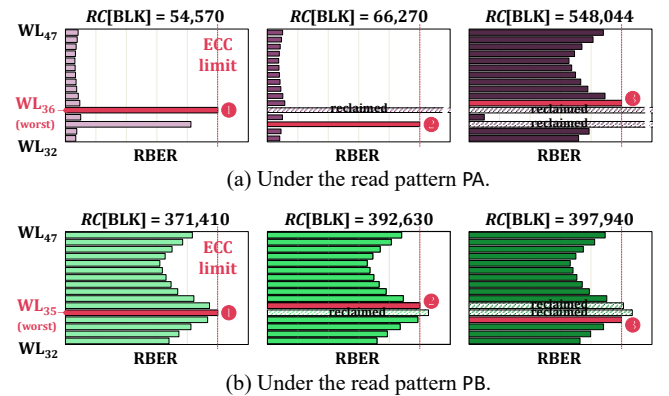


Figure 7. WL-level read-disturbance management by WR^2 .

481,774 reads before reclaiming WL₃₇. Similarly, under the PB pattern, which reads all valid pages in the block sequentially, WR² reclaims the WLs in order of the accumulated read-disturbance impact. As shown in Figure 7(b), WR² first reclaims WL₃₅ after 371,410 RC, followed by WL₃₆ and WL₃₄ after 392,630 and 397,940 RC, respectively. As explained in §3.2, to avoid data loss in the worst-case access pattern (the pattern PA), the existing block-level RR must set RC_{MAX} to 54,570 and reclaim all valid pages in the block after 54,570 RC in both patterns. In contrast, this WL-level RR approach not only ensures data reliability but also allows a significant number of additional reads to the block by deferring WL reclamation until necessary, such as reclaiming WL₃₇ after 548,044 reads, as shown in Figure 7(a).

A key challenge for WR² is accurately determining whether a WL is heavily disturbed. To address this, WR² employs (i) a read-disturbance model and (ii) per-WL counters. The model quantifies two key factors of read-disturbance variations in high-density 3D NAND flash memory. First, to account for inherent process variations across WLs [47, 55, 66, 67, 73, 78, 79], the model defines a per-WL threshold ERC_{MAX}—the maximum number of effective reads a WL can tolerate without incurring uncorrectable errors. For WL_i, ERC[WL_i] is the cumulative number of default- V_{pass} applications to WL_i since it was last programmed. Second, to quantify V_{pass} -dependent read-disturbance asymmetry, the model defines the disturbance rate α as the ratio of per-read disturbance stress under the higher V_{passH} (applied to WLs adjacent to the target WL) to that under the default V_{pass} . Per-WL counters record the read count RC for each WL and thereby determine how many times V_{passH} and V_{pass} were applied to that WL.

Figure 8 depicts a high-level overview of WR²'s decision procedure. To determine whether WL_i is heavily disturbed, WR² (1) scales the number of V_{passH} applications by the disturbance rate α to obtain V_{passH} 's contribution to ERC of WL_i; (2) sums the contributions from V_{passH} and V_{pass} to compute the total ERC of WL_i; and (3) compares this total ERC against ERC_{MAX} of WL_i to decide whether WL_i should be reclaimed.

Feasibility & Overhead. WR² only requires simple modifications to the existing SSD firmware but no modification to the flash controller and chips. However, WR² has two potential drawbacks. First, a naïve implementation of per-WL counters introduces non-trivial storage overhead. For example, assuming a 2 TB SSD comprising a block with 2,568

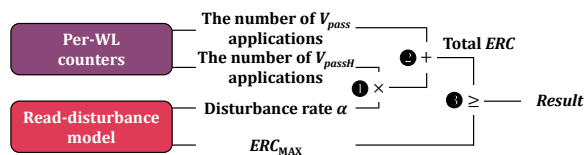


Figure 8. A high-level overview of how WR² determines whether a WL is heavily disturbed.

WLs [34], the per-WL counters require approximately 125 MB of internal DRAM space (one 3-byte counter per 48 KB WL), which is more than 2,568 times the space required for per-block counting. We address this drawback with a space-optimized implementation of per-WL counters in §6.

Second, WL-level RR may affect block erasure counts by introducing additional garbage collection (GC) invocations. Since WR² selectively reclaims only the heavily disturbed WLs within a block, block erasure is deferred until all WLs exhibit significant disturbance. Consequently, under a write-intensive workload, GC operations may be triggered more frequently to create sufficient free blocks compared to conventional block-level RR. We evaluate this drawback in terms of total block erasures in §7.

4.2 Stress-Reduced Read (SR²)

Key Idea. Since the reliability impact of read disturbance is proportional to V_{pass} [25], reducing V_{pass} is highly effective in mitigating stress on non-target WLs. However, applying a reduced V_{pass} can increase bit errors—non-target WLs may fail to properly pass data from the target WL to the sensing circuitry.

To mitigate these side effects, prior works on 2D NAND flash memory [9, 25] have proposed reducing V_{pass} to non-target WLs only when sufficient error margin exists; i.e., when the ECC correction capability sufficiently exceeds the RBER of the target page. However, we find that such techniques are difficult to apply to modern SSDs due to two key challenges. First, V_{pass} reduction can substantially affect the read output's RBER in high-density 3D NAND flash memory, which is inherently more error-prone than conventional 2D NAND flash memory. Such vulnerability often leaves insufficient error margins to safely apply V_{pass} reduction under many operating conditions, thereby limiting its applicability. Second, even when sufficient error margins are available, reducing V_{pass} on non-target WLs can significantly increase the latency required to reliably read the target WL. As bit errors on the target page increase, read latency can rise by several orders of magnitude due to prolonged ECC decoding or the need for additional read-retry steps [19]. For example, naively reducing V_{pass} on all non-target WLs by 10% increases the number of bit errors per 1 KB of read data from 18 to 84 at 1K P/E cycles, inflating ECC decoding time by 5×.

SR² addresses these challenges by applying a *higher* V_{pass} to *invalid* WLs—i.e., WLs containing only invalid pages—while applying a reduced V_{pass} to valid WLs. Our key insights are that (i) applying a higher V_{pass} to certain non-target WLs can reduce the RBER of the target WL, and (ii) the increased read-disturbance impact on invalid WLs does not affect overall data reliability. SR² exploits the available error margins from the selective application of a higher V_{pass} to invalid WLs to safely reduce V_{pass} for valid WLs. However, when a block contains too few invalid WLs to provide sufficient error margin, SR² disables V_{pass} reduction. By doing so, SR²

avoids introducing additional bit errors from V_{pass} reduction, thereby preventing side effects such as extended read latency.

Figure 9 illustrates how SR^2 mitigates read-disturbance stress on valid WLs. During a read operation, the default V_{pass} levels are 6.5 V for adjacent WLs and 7.2 V for non-adjacent WLs. Under these settings, N_{ERR} for WL_4 —defined as the maximum number of bit errors per 1 KB of data across pages in WL_4 —is 23 (Figure 9(a)). In this example, since WL_1 , WL_2 , WL_3 , and WL_5 contain only invalid data, SR^2 increases the V_{pass} of the two adjacent invalid WLs (WL_3 , WL_5) from 7.2 V to 7.9 V and the two non-adjacent invalid WLs (WL_1 , WL_2) from 6.5 V to 7.2 V (Figure 9(b)). This adjustment reduces N_{ERR} by 13 (from 23 to 10). SR^2 then leverages this error margin to lower V_{pass} by 0.2 V on the two valid WLs (WL_6 , WL_7) (Figure 9(c)), thereby mitigating read-disturbance stress on these valid WLs without incurring additional N_{ERR} .

SR^2 is tightly integrated with WR^2 in two ways. First, WR^2 's WL-level RR maximizes the applicability of SR^2 . Once WR^2 reclaims heavily disturbed WLs in a block, these WLs become invalid, enabling subsequent reads on the remaining WLs to benefit from a more aggressive V_{pass} reduction. Second, we extend the read-disturbance model to account for the reduced stress from applying a lower V_{pass} . This extension allows WR^2 to reclaim only the heavily disturbed WLs at the right time, even when SR^2 is active. The details of the extended read-disturbance model is presented in §5.4.

Feasibility & Overhead. Applying SR^2 to commodity NAND flash chips is highly feasible at low cost. First, conventional chips already incorporate WL switching circuitry that can bias each WL in a block with a designated V_{pass} level: for example, when reading a WL, the chip applies V_{passH} to its neighboring WLs—regardless of their physical locations. Second, they employ multiple V_{pass} sources for several purposes—for instance, to reduce WL setup time during read operations [34, 50]. These V_{pass} sources enable per-WL precharge to a distinct V_{pass} level within a block. Third, we expect that these V_{pass} levels are programmable via existing post-fabrication tuning mechanisms. Due to strong process variation, the internal operating parameters of each chip must be carefully tuned after fabrication before it can be shipped as a commercial product [36]. In our test chip, all such parameters—including V_{pass} for individual WLs—can be modified by programming electrical fuses placed in the peripheral

circuit. Since similar post-fabrication tuning mechanisms are commonly used in commodity 3D NAND chips, we expect comparable per-WL tuning capabilities to exist in other devices as well (e.g., from Samsung [36] and Seagate [65]).

SR^2 inevitably increases read-disturb stress on invalid WLs during read operations. This side effect, however, is limited to invalid WLs: in commodity SSDs, once a page is marked invalid, its content is either rewritten to a different physical location or explicitly deleted, and thus pages on invalid WLs are no longer accessed. Consequently, it neither raises reliability concerns nor increases future RR overhead.

In SR^2 , the precharge phase of read operations may increase latency and energy, as invalid WLs must be precharged to a higher V_{pass} than the default. In terms of latency, however, existing chips typically apply an overdrive voltage during the precharge phase [5, 50] that is already significantly higher than the V_{pass} levels targeted by SR^2 , making the impact on read latency practically negligible. We quantitatively evaluate this effect in §7. In terms of energy, when SR^2 applies a 10% higher-than-default V_{pass} to invalid WLs, the precharge energy can increase by up to 10% in the worst case where all WLs in a block are invalid except the target WL. Because precharge contributes $\approx 25\%$ of a read operation, SR^2 would increase the read energy by $\approx 2.5\%$ in this worst case. However, when SR^2 applies a lower V_{pass} to valid WLs, the precharge energy decreases, so the typical energy overhead is smaller than this worst-case bound. Furthermore, because SR^2 reduces RR-induced page copies, we expect the total energy consumption of an SR^2 -enabled SSD to be lower than that of existing SSDs.

5 Device Characterization Study

We conduct an extensive real-device characterization study to validate our key ideas and hypotheses in §4.

5.1 Characterization Methodology

Infrastructure. We use an FPGA-based testing platform equipped with a custom flash controller. The controller supports not only basic NAND-flash commands (i.e., read, program, and erase), but also low-level test commands, such as SET FEATURES [54]. This command allows us to adjust V_{pass} for each read operation at WL granularity. We characterize 160 3D TLC NAND flash chips, which are manufactured using 3D charge-trap technology [17, 30]. Although our study is based on empirical characterization of a specific flash chip generation, it is widely assumed that different 3D flash chips share key device-level characteristics due to their structural and cell-type similarities [15, 37, 58]. Therefore, we believe that our characterization results will follow the general trends observed in other 3D flash chips.

Methodology. In our experiments, we follow the JEDEC standards [31, 32], which specify the test methodology recommended for evaluating the reliability of commercial-grade

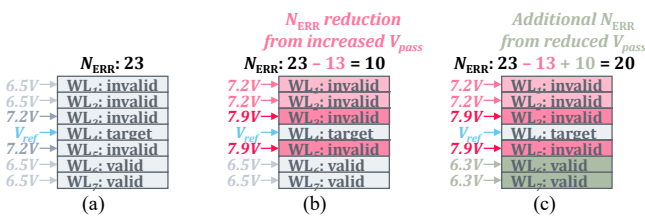


Figure 9. Overview of the SR^2 mechanism.

NAND flash products. For instance, to satisfy the JEDEC guideline of testing more than 39 chips from three different wafers, we use 160 TLC 3D NAND flash chips from five different wafers. To capture various variations of 3D NAND flash memory [47, 55, 66, 67, 73, 78, 79] in a statistically significant manner, we test a total of 3,686,400 WLs (11,059,200 pages). Specifically, we evenly select 120 blocks from each of the 160 chips at different physical block locations and measure the RBER of every page in each selected block. Unless specified otherwise, we report a representative (i.e., worst) value across all the tested pages from the 160 chips.

5.2 Read-Disturbance Model

WR² employs a new read-disturbance model that quantifies (i) the maximum number of effective reads (ERC_{MAX}) and (ii) the disturbance rate (α). To derive this model, we measure the maximum read count for each WL across the tested blocks while varying PEC and read patterns. While defining ERC_{MAX} and α for each WL individually would be ideal, we instead classify WLs within a block into four groups—Best, Good, Bad, and Worst—based on their initial RBER. This WL-group granularity model significantly reduces characterization overhead while maintaining high accuracy due to the similar error characteristics of WLs within the same group; for instance, as shown in Figure 4(b), the maximum number of tolerable reads varies by 559K across all WLs in a block but only by 144K within a group.

Figure 10 compares the maximum read counts of four WL groups under different PECs and read patterns. For each WL group, a coordinate (x, y) indicates that the group's worst WL tolerates up to y page reads on the same block (i.e., read disturbance from the reads) given $x\%$ adjacent-WL reads and $(100 - x)\%$ non-adjacent-WL reads. We make three key observations. First, reading a WL causes significantly more disturbance ($8.4\times$ on average) to adjacent WLs compared to non-adjacent WLs. Second, under the same operating condition, the ratio of read disturbance impact between non-adjacent and adjacent WL reads maintains a consistent disturbance rate α . For instance, at 2K PEC, the disturbance rate α is 8.7 for the Best group, indicating that reading the adjacent WL of the worst WL in the Best group (WL_w^{Best}) incurs $8.7\times$ more disturbance stress on WL_w^{Best} compared to reading the non-adjacent WL of WL_w^{Best} . Third, the read-disturbance

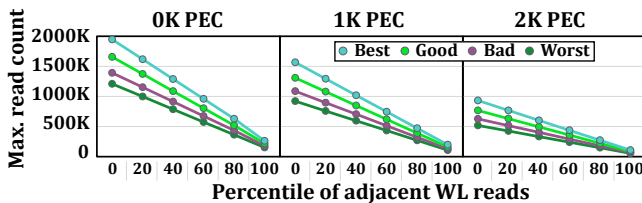


Figure 10. Comparisons of the maximum read counts of each WL Group at different PECs under various read patterns.

	ERC_{MAX}	α	ERC_{MAX}	α	ERC_{MAX}	α	ERC_{MAX}	α
0	1947K	7.4	1657K	7.6	1391K	7.8	1210K	8.0
0.5K	1806K	7.7	1521K	7.9	1270K	8.0	1094K	8.3
1K	1567K	7.9	1309K	8.2	1087K	8.4	922K	8.6
1.5K	1310K	8.3	1086K	8.6	899K	8.8	749K	9.0
2K	933K	8.7	767K	9.0	627K	9.2	518K	9.5
2.5K	539K	9.2	438K	9.5	354K	9.7	288K	10.1
3K	111K	9.7	89K	10.0	71K	10.3	58K	10.7
	Best		Good		Bad		Worst	

Figure 11. Final Model of ERC_{MAX} and α .

tolerance of a WL group and the disturbance rate α vary significantly depending on operating conditions, such as PEC and the inherent reliability characteristics of the WLs.

Figure 11 shows the parameters of the final model for the tested flash chips under different PECs. For example, at 2K PEC, WL_w^{Good} can tolerate 767K non-adjacent reads, and the disturbance rate α is 9.0, respectively.

5.3 V_{pass} vs RBER of the target WL

To validate the feasibility of SR², we quantify how adjusting V_{pass} on non-target WLs affects N_{ERR} . We measure N_{ERR} of the target WL while varying V_{pass} on (i) non-adjacent WL subsets $\{25\%, 50\%, 75\%\}$ ³, and (ii) two adjacent WLs. We apply the default V_{pass} to all non-selected WLs— $V_{pass} = 6.5$ V for non-adjacent WLs and $V_{passH} = 7.2$ V for adjacent WLs.

Figure 12 depicts N_{ERR} as a function of ΔV_{pass} (the offset from the default V_{pass}), applied to (i) non-adjacent WL subsets and (ii) the two adjacent WLs. For non-adjacent subsets, ΔV_{pass} ranges from -10% to 10% in 2.5% increments; for adjacent WLs, ΔV_{pass} from -20% to 20% in 5% increments. We make three key observations. First, it is possible to reduce N_{ERR} by increasing V_{pass} on non-target WLs. For example, when $\Delta V_{pass} = 10\%$ is applied to 75% of the non-adjacent WLs, N_{ERR} is reduced by 33.3% and 39.1% at 1K and 2K PEC, respectively. Second, the decrease in N_{ERR} with positive ΔV_{pass} saturates. At 1K PEC, setting $\Delta V_{pass} = 10\%$ on the adjacent

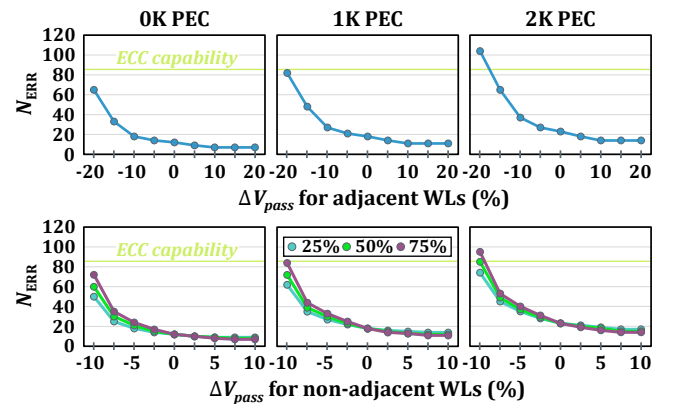


Figure 12. Impact of V_{pass} scaling on N_{ERR} .

³We randomly select subsets of non-adjacent WLs; we observe that the locations of the selected non-adjacent WLs do not measurably affect N_{ERR} .

WLs lowers N_{ERR} by 33% relative to the default V_{pass} , whereas further increasing ΔV_{pass} to 20% provides only an additional reduction of 8.3%. Third, ΔV_{pass} on the two adjacent WLs has a greater impact on N_{ERR} than ΔV_{pass} on non-adjacent WLs. As explained in §3.2, during a read operation, a modern flash chip applies a higher V_{passH} to the two adjacent WLs. We hypothesize that such a design leverages the observed asymmetry: applying V_{passH} only to the adjacent WLs keeps N_{ERR} low while confining substantial V_{passH} -induced read-disturb stress to those two WLs.

From these observations, we draw three conclusions. First, SR^2 can safely lower V_{pass} on valid WLs when a block contains sufficient invalid WLs. Consider a block where 50% of non-adjacent WLs and the two adjacent WLs are invalid. At 1K PEC, setting $\Delta V_{pass} = 10\%$ on all invalid WLs reduces N_{ERR} by 72.2%, enabling a 5% reduction in V_{pass} on 50% of the remaining valid WLs without increasing N_{ERR} . Second, in the SR^2 design, we limit the V_{pass} increase on invalid WLs to 10%. As shown in Figure 12, the reduction in N_{ERR} from positive ΔV_{pass} saturates quickly; increases in V_{pass} on invalid WLs beyond 10% offer negligible additional benefit. Third, we exclude lowering V_{pass} on adjacent WLs—even when they are valid—because the cost dominates the benefit; instead, SR^2 allocates the available N_{ERR} margin to reducing V_{pass} on invalid non-adjacent WLs. Although lowering V_{pass} on adjacent WLs mitigates read-disturb stress only for those two WLs, a 5% reduction increases N_{ERR} by up to 17.3%.

5.4 V_{pass} -Reduction Model

Based on the characterization results, we develop a V_{pass} -reduction model that defines the minimum V_{pass} for valid WLs, given (i) the validity of the two adjacent WLs and (ii) the fraction of invalid non-adjacent WLs (R_{INV}). Figure 13 shows the final model. Each cell shows two values: the N_{ERR} margin (gray) and the maximum allowable $-\Delta V_{pass}$ for valid WLs (green); hatched entries (red) indicate that no V_{pass} reduction is allowed. When the two adjacent WLs are invalid and $R_{INV} = 0.8$, for example, SR^2 can lower V_{pass} on valid WLs by 5% by exploiting N_{ERR} margins of 9, 13, and 17 bits at 0K, 1K, and 2K PEC, respectively.

As the final step in our characterization, we extend the read-disturbance model to incorporate V_{pass} reduction. We first measure ERC_{MAX} on the worst WL in each of the four WL groups in Figure 11, with ΔV_{pass} set to 0%, -2.5%, and -5%. We then derive two disturbance rates, $\beta_{2.5}$ and β_5 , for

R_{INV}	Error margins				Allowable $-\Delta V_{pass}$ (%)			
	OK PEC		1K PEC		1K PEC		2K PEC	
< 0.25	0	5	0	7	0	9	0	9
< 0.50	3	8	4	11	6	15	2.5	15
< 0.75	4	2.5	9	5	6	2.5	13	5
< 1.00	5	2.5	10	5	7	2.5	14	5
	valid	invalid	valid	invalid	valid	invalid	valid	invalid

Figure 13. Final V_{pass} Reduction Model.

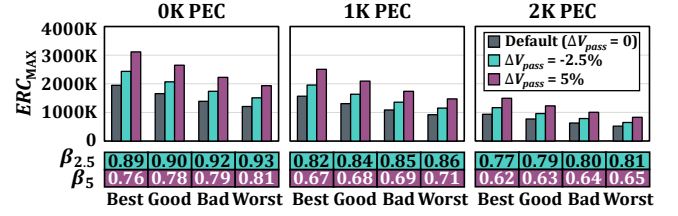


Figure 14. Impact of V_{pass} reduction on ERC_{MAX} and the extended read-disturbance model parameters.

each $\langle \text{PEC}, \text{WL group} \rangle$ pair from the measured ERC_{MAX} values, where β_i denotes that reading a page with V_{pass} lowered by $i\%$ incurs $\beta_i \times$ the read-disturbance stress relative to the default V_{pass} . Figure 14 compares ERC_{MAX} , $\beta_{2.5}$, and β_5 for each $\langle \text{PEC}, \text{WL group} \rangle$ pair. We conclude that SR^2 is highly effective at improving read-disturbance tolerance: ERC_{MAX} increases exponentially as V_{pass} decreases. For example, at 2K PEC, setting $\Delta V_{pass} = -5\%$ reduces per-read disturbance stress on the worst WL in WL_w^{Best} by 38% ($\beta_5 = 0.62$), allowing 59.1% more page reads on the target block.

6 Design and Implementation of STRAWFTL

We implement STRAWFTL, an STRAW-enabled FTL, by extending a conventional page-level FTL [41] with three key data structures: (i) Read-reclaim Parameter Table (RPT), (ii) Resource-Efficient Counters (REC), and (iii) Pass-through Voltage Table (PVT). RPT stores ERC_{MAX} and disturbance rates α , $\beta_{2.5}$, and β_5 for each $\langle \text{PEC}, \text{WL group} \rangle$ pair; we construct it via offline profiling of the target chips (Figure 11 and Figure 14). REC maintains per-block counters that track the RC of each WL as well as the block-level RC. PVT records the minimum V_{pass} for each tuple $\langle \text{PEC}, \text{validity of the two adjacent WLs}, R_{INV} \rangle$ (Figure 13).

Operational Overview. Figure 15 depicts STRAWFTL’s operational overview for WL-level read-disturbance management with WR^2 and SR^2 enabled. Whenever a page is read from WL_{tgt} in BLK_{tgt} —the target WL and its containing block—STRAWFTL first selects the minimum-safe V_{pass} mode for non-adjacent WLs. It then updates the REC entries for both BLK_{tgt} and WL_{tgt} . Every predefined interval (e.g., at each 1K reads to the block), STRAWFTL checks all valid WLs in BLK_{tgt} to determine whether the accumulated disturbance impact on any valid WL exceeds the threshold or if there is a possibility it will exceed the threshold by the next interval. For such WLs, STRAWFTL copies the valid pages to

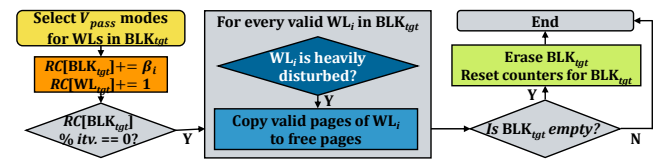


Figure 15. Operational overview of STRAWFTL.

free pages before the next interval, thereby preventing read-disturbance-induced data corruption. If BLK_{tgt} contains no valid pages after the checking procedure, STRAWFTL erases BLK_{tgt} and resets all associated counters.

V_{pass} Adjustment. To enable SR^2 for each read, the STRAWFTL and flash chips interact through a three-step procedure. First, it derives R_{INV} and the validity of the two adjacent WLs from the valid bitmap of BLK_{tgt} .⁴ It then queries PVT with $\langle PEC, \text{adjacent-WL validity}, R_{INV} \rangle$ to select the minimum-safe V_{pass} for the valid non-adjacent WLs. Second, it programs the target chip by issuing a SET FEATURES command [54] that encodes the per-WL V_{pass} values.⁵ The chip interprets the command, decodes its fields, and applies WL-specific V_{pass} —lower on valid non-adjacent WLs, as determined by PVT, and 10% higher on invalid WLs. Third, STRAWFTL updates the REC entry for BLK_{tgt} in a β_i -weighted manner based on the applied V_{pass} ; for example, at 2K PEC, with WL_{tgt} in the Best WL group, if $\Delta V_{pass} = -5\%$ (i.e., $\beta_5 = 0.62$), it increments that entry by 0.62 to reflect the reduced read-disturbance stress on non-adjacent WLs.

Identifying Heavily Disturbed WLs. Figure 16 illustrates how STRAWFTL estimates the accumulated disturbance impact on individual WLs. For WL_i , which is located in the i -th WL in the k -th block, it first looks up the RC values of WL_{i-1} , WL_i , WL_{i+1} , and BLK_k from REC (❶). Based on the obtained RC values, STRAWFTL determines the number of reads to WL_i 's adjacent and non-adjacent WLs (❷). Then it queries RPT with the PEC of BLK_k and the WL group to which WL_i belongs (❸). STRAWFTL converts the number of reads to adjacent and non-adjacent WLs of WL_i into the ERC, using the disturbance rate α from the query result (❹). If the ERC of WL_i (including the possible additional reads by the next interval) exceeds ERC_{MAX} from RPT, STRAWFTL identifies WL_i as a heavily disturbed WL (❺).

6.1 Overhead Optimization

STRAWFTL requires simple modifications over the conventional FTL at a high level, but—as explained in §4.1—naïve per-WL counters incur non-negligible storage overhead. To

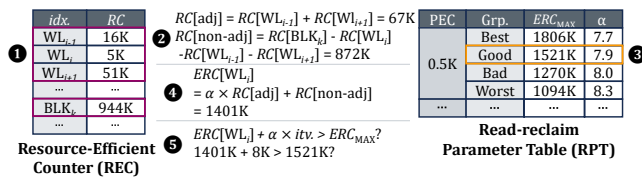


Figure 16. A procedure for identifying heavily disturbed WLs in STRAWFTL.

⁴Modern SSD FTLs typically maintain a per-block valid bitmap to enable efficient garbage collection [41].

⁵A SET FEATURES command provides 158 vendor-specific feature addresses [54], each with four 8-bit parameters, which is sufficient to encode four modes per WL, using only 642 bits (21 addresses) for a 321-WL chip.

minimize the storage overhead of per-WL counters, REC incorporates the Space-Saving (SS) algorithm [49], which efficiently estimates the frequency of elements in a data stream using a limited number of counters. Each counter entry consists of an element index and its corresponding count value. In our context, the data stream consists of a sequence of reads to a block between successive block erases.

Upon block erasure, REC initializes a predefined number of counter entries for the erased block. Whenever a WL is read, REC first checks for an existing counter entry associated with the target WL index. If an entry exists, REC increments the corresponding count value. If no entry exists, REC identifies the entry with the *lowest* count value, increments its count value, and replaces the element index with the current read WL index. When STRAWFTL queries the read count for a particular WL, REC returns the corresponding count value if an entry exists. If no entry is found, REC returns the minimum count value among all entries. Due to the limited number of counters, the estimated read counts by REC may introduce some error, but SS ensures that the estimated count value for any element is never underestimated [49]. This guarantees that errors in estimation do not result in read-disturbance-induced data corruption, although they may cause premature RR invocations.

7 Evaluation

We evaluate the effectiveness of STRAW in improving the lifetime and performance of modern SSDs.

7.1 Evaluation Setup

For our experiments, we extend NVMeVirt [40], a state-of-the-art SSD emulator, to faithfully evaluate the effectiveness of STRAW. Specifically, we extend NVMeVirt to (i) trigger RR based on read-disturbance characteristics—e.g., maximum tolerable read counts at different PECs—we measured measured from 3,686,400 WLs across the tested chips and (ii) model N_{ERR} -dependent read latency by incorporating read-retry loops and ECC decoding time.

We configure the architecture and key parameters of the emulated SSDs to closely match modern SSDs employing high-density 3D TLC NAND flash memory [14, 29, 64, 69]. In addition, to capture the key microarchitectural latencies relevant to STRAW, we (i) set NVMeVirt's read-latency parameter to incorporate SR^2 's precharge overhead and (ii) calibrate NVMeVirt's configurable controller-level latency parameter to account for the transfer latency of the SET FEATURES command used to enable SR^2 (22 ns for 352-bit feature data plus 30 ns for the opcode and feature address). Table 1 summarizes the emulated SSD configuration.

We evaluate seven workloads: five from three benchmark suites—FIO [3], Yahoo! Cloud Serving Benchmark (YCSB) [20], and Filebench (FB) [48]—and two real-world applications:

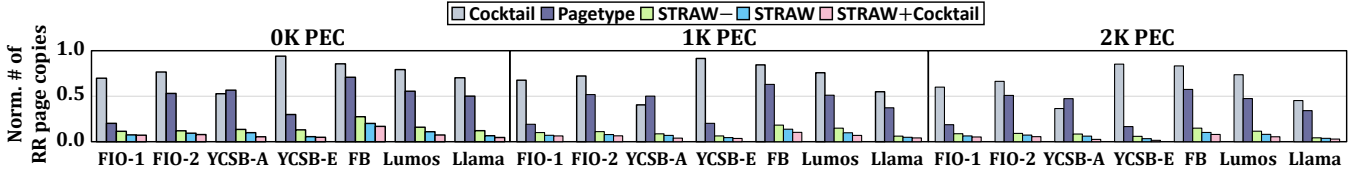


Figure 17. Comparison of RR-induced page copies under three different PECs.

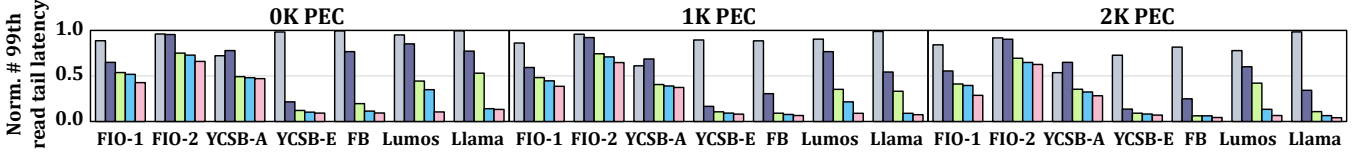


Figure 18. Comparison of 99th percentile read tail latencies under three different PECs.

Lumos (graph processing) [72] and Llama (large-language-model processing) [71]. Table 2 summarizes the representative I/O characteristics of the evaluated workloads.

We evaluate six SSD configurations: Baseline, Cocktail, Pagetype, STRAW–, STRAW, and STRAW+Cocktail. Baseline serves as the baseline SSD: it triggers block-level RR at a conservatively set RC_{MAX} . To faithfully evaluate the impact of RR on read latency, Baseline employs two mechanisms: (i) a priority-aware scheduler that prioritizes user read requests over RR-induced operations [18, 70]; and (ii) erase suspension to reduce tail latency from erase–read conflicts [39, 74]. Cocktail and Pagetype extend Baseline with state-of-the-art RR optimizations [26, 81]. During RR, Cocktail [81] redistributes hot pages across blocks to balance read counts, thereby reducing subsequent RR invocations. Pagetype [26] implements page-type-level RR: it classifies pages within each block by type (e.g., MSB/CSB/LSB in TLC) and migrates them based on their susceptibility to read disturbance. STRAW– is an SSD that extends Baseline with WR^2 , whereas STRAW implements both of our techniques— WR^2 and SR^2 . We implement STRAW+Cocktail—which integrates STRAW with Cocktail—to evaluate STRAW’s effectiveness when combined with a state-of-the-art RR optimization. By default, STRAWFTL in

STRAW–, STRAW, and STRAW+Cocktail uses a 32-entry REC per 704-WL block.

7.2 Performance Evaluation

RR Efficiency. We first measure the number of page copies induced by RR, which directly indicates the effectiveness of read-disturbance management. Figure 17 compares the number of RR-induced page copies in six SSDs, normalized to Baseline, under three different PECs.

We make three observations. First, STRAW significantly improves RR efficiency in read-intensive workloads by (i) minimizing the number of premature RR invocations and (ii) proactively mitigating per-read read-disturbance stress. STRAW reduces the number of RR-induced page copies by (90%, 92.2%, 93.6%) on average compared with Baseline at PEC = {0K, 1K, 2K}. Second, Cocktail and Pagetype reduce RR-induced page copies; however, their effectiveness remains limited relative to STRAW. Pagetype addresses the heterogeneous impact of read-disturbance across page types (e.g., LSB pages are 1.3× more vulnerable [26]) by setting page-type-level RR thresholds; however, to ensure reliability under the worst-case read pattern, these thresholds must be set significantly lower than the WL-level *ERC*. Similarly, Cocktail’s effectiveness is pattern-dependent: it is particularly effective on workloads with strong temporal locality (e.g., YCSB-A). Third, STRAW+Cocktail further improves read-disturbance management efficiency. Compared with STRAW, it reduces RR-induced page copies by up to 31% (25.8% on average) on read-dominant workloads (read ratio > 0.8). This suggests that STRAW can be combined with existing RR optimizations to mitigate RR’s negative impact, which is expected to worsen in future NAND flash memory.

Read Tail Latency. We evaluate the impact of reducing RR invocations on read tail latency—a crucial performance metric for modern data-intensive applications [18, 27, 46, 76, 82, 84]. Figure 18 compares the 99th-percentile read

Table 1. Evaluated SSD configurations.

Configuration	480 GB capacity; 8 channels; 2 dies/channel; 4 planes/die; 176 layers/block; 4 WLs/layer; 232 blocks/plane; 2112 pages/block
Latencies	tREAD = 40 μ s; tPROG = 380 μ s; tERASE = 3500 μ s; tSF = 52 ns;
Interface	8.0 GB/s external I/O bandwidth (PCIe 4.0, 4-lane); 2.0 GB/s channel I/O bandwidth

Table 2. Key I/O characteristics of seven workloads.

Workload	FIO-1	FIO-2	YCSB-A	YCSB-E	Filebench	Lumos	Llama
Read ratio	0.98	0.71	0.7	0.97	0.98	0.91	0.99
Read pattern	Rand.	Rand.	Mixed.	Rand.	Seq.	Rand.	Seq.

latencies across the six SSDs at three different PECs; all values are normalized to Baseline. We observe that reducing RR invocations significantly lowers the 99.9th-percentile read latency. Relative to Baseline, STRAW (STRAW-) achieves $\langle 65.2\%, 71.1\%, 75.6\% \rangle$ ($\langle 56.1\%, 64\%, 69.4\% \rangle$) reductions in the 99.9th-percentile read latency at PEC = $\langle 0K, 1K, 2K \rangle$. In particular, STRAW is highly effective on YCSB-E and Llama because their high read intensity often precludes deferring RR to serve user reads first. These results highlight the need for efficient read-disturbance management—such as our proposal—to meet the strict service-level agreements (SLAs) of data-intensive applications [1].

SS Counter’s size vs RR Efficiency. We evaluate how STRAW’s benefits change depending on the size of REC. When the REC size is limited relative to the workload’s working-set size, it frequently overestimates counter values, triggering premature RR. Figure 19 compares the number of RR-induced page copies for STRAW at 1K PEC across REC sizes—8, 16, 32, and 64 entries per 704-WL block—for five read-dominant workloads. All values are normalized to an REC without space constraints—i.e., per-WL counters.

We make two observations. First, integrating SS counters into REC reduces space overhead—at a size-dependent cost to RR efficiency. 32-entry and 64-entry per-block REC reduce space overhead by 95.5% and 91%, respectively, at the cost of 74.4% and 26.6% additional RR-induced page copies. Second, REC efficiency is pattern-dependent: SS counters, which are optimized to accurately track a small set of hot entries [21, 22], are effective for random-read workloads with stable hotspots (e.g., Lumos) but degrade under sequential workloads without stable hotspots (e.g., YCSB-E).

7.3 Overhead Evaluation

Impact on Lifetime. As discussed in §4.1, WR^2 ’s WL-level RR can increase the frequency of future GC invocations. Unlike block-level RR, which immediately frees a block after an RR invocation, WR^2 defers block erasure until all WLs in the block are heavily disturbed. This significantly reduces premature RR invocations but, under mixed read/write workloads, necessitates additional GC to maintain a sufficient number of free blocks. To quantify WR^2 ’s impact on SSD lifetime—including GC overhead—we measure the total block-erasure count for Baseline and STRAW across two mixed read/write workloads (FIO-2, YCSB-A). Figure 20(a) compares total block-erasure counts for the two SSDs, normalized to Baseline, at three PECs. STRAW reduces total block erasures

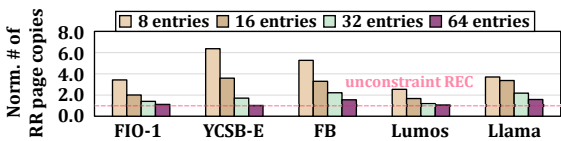


Figure 19. Comparison of RR efficiency as REC size varies.

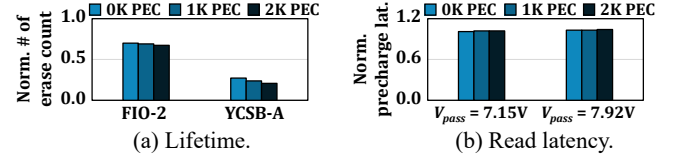


Figure 20. Overhead analysis of STRAW.

across all tested conditions (53.6% on average), demonstrating that lifetime gains from preventing premature RR outweigh the additional GC cost—even under mixed workloads.

Impact on Read Latency. We evaluate the impact of SR^2 —increasing V_{pass} on invalid WLs 10% above the default—on read latency. To this end, we measure the precharge latency required to charge V_{pass} to 7.92 V and 7.15 V—the levels that SR^2 applies to invalid adjacent and non-adjacent WLs, respectively. Figure 20(b) depicts the worst-case precharge latency at three PECs. As shown in the figure, SR^2 increases precharge latency by at most 4%; since precharge constitutes $\approx 30\%$ of total read latency [58], the overall impact is only $\leq 1.2\%$, making SR^2 ’s read-latency overhead negligible.

8 Related Work

We already discussed and compared to the state-of-the-art techniques closely related to STRAW (§4.2 and §7). In this section, we briefly review related read-disturbance management techniques at a high level.

RR Optimization. Many prior studies have optimized RR to improve SSD lifetime and performance [25, 26, 35, 43, 45, 68, 81, 83]. State-of-the-art techniques for modern SSDs [45, 68, 81, 83]—including the baseline technique used in our evaluation [81]—primarily focus on redistributing read-hot pages across blocks during RR to reduce subsequent RR invocations. Unlike these techniques, our proposal—motivated by the heterogeneous read-disturbance impact on non-target WLs in high-density 3D NAND flash memory—prevents premature RR by rethinking RR granularity. Moreover, STRAW can integrate readily with existing techniques to further improve SSD lifetime and performance, as discussed in §7.

Pass-Through Voltage Scaling. To our knowledge, only a few prior works [9, 25] have explored V_{pass} -scaling techniques for 2D NAND flash memory to mitigate read disturbance. Ha et al. [25] introduce an integrated approach that programs read-hot blocks with fine-grained control to form narrow V_{th} distributions and uses a lower V_{pass} for reads in those blocks. Cai et al. [9] propose lowering V_{pass} when sufficient ECC margin exists in the target block. Although effective on conventional SSDs, these techniques have not been evaluated for side effects on error-prone, high-density 3D NAND flash memory-based SSDs, as discussed in §4.2. In contrast, we demonstrate that SR^2 safely mitigates read-disturbance effects in high-density SSDs without incurring side effects.

9 Conclusion

This work presents STRAW, a novel read-disturbance management technique that significantly improves the lifetime and performance of modern SSDs by reducing RR invocations. By identifying heavily disturbed WLs within each block and reclaiming them at the WL-level, STRAW avoids the premature RR invocations that existing block-level techniques inevitably incur. To further reduce RR invocations, STRAW employs an intelligent V_{pass} -scaling mechanism that mitigates read-disturbance effects on non-target WLs without incurring side effects. Our evaluations with a STRAW-aware SSD show that STRAW effectively mitigates the negative impact of read disturbance on high-density SSDs.

Acknowledgments

We thank our shepherd Huaicheng Li and anonymous reviewers of ASPLOS 2026 for their valuable feedback and comments. This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) (RS-2024-00347394, RS-2024-00456287, RS-2025-02214654, and RS-2025-02314443). Myungsuk Kim was supported by Korea Collaborative & High-Tech Initiative for Prospective Semiconductor Research (02219428, 2410012261, and 25066-15FC) and National Research Foundation of Korea (NRF) (NRF-2022R1I1A3073170 and RS-2024-00414964). Jisung Park was supported by IITP (RS-2024-00437866) and NRF (RS-2024-00415602 and RS-2025-00519994). The ICT at Seoul National University provided research facilities for this study. Jihong Kim and Myungsuk Kim are the corresponding authors.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *OSDI*.
- [2] Sangmin Ahn, Hyungjun Jo, Sechun Park, Jongwoo Kim, and Hyungcheol Shin. 2025. Investigation of Cell Variation Effect on Z-Interference in Charge-Trip-Based 3-D NAND Flash Memory. *IEEE T-ED* (2025).
- [3] Jens Axboe. 2006. FIO: Flexible I/O Tester. <https://github.com/axboe/fio>.
- [4] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. 2017. Error Characterization, Mitigation, and Recovery in Flash-Memory-Based Solid-State Drives. *Proc. IEEE* (2017).
- [5] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. 2018. Reliability Issues in Flash-Memory-Based Solid-State Drives: Experimental Analysis, Mitigation, Recovery. In *Inside Solid State Drives* (2nd ed.). Springer.
- [6] Yu Cai, Saugata Ghose, Yixin Luo, Ken Mai, Onur Mutlu, and Erich F. Haratsch. 2017. Vulnerabilities in MLC NAND Flash Memory Programming: Experimental Analysis, Exploits, and Mitigation Techniques. In *HPCA*.
- [7] Yu Cai, Erich F. Haratsch, Onur Mutlu, and Ken Mai. 2012. Error Patterns in MLC NAND Flash Memory: Measurement, Characterization, and Analysis. In *DATE*.
- [8] Yu Cai, Erich F. Haratsch, Onur Mutlu, and Ken Mai. 2013. Threshold Voltage Distribution in MLC NAND Flash Memory: Characterization, Analysis, and Modeling. In *DATE*.
- [9] Yu Cai, Yixin Luo, Saugata Ghose, and Onur Mutlu. 2015. Read Disturb Errors in MLC NAND Flash Memory: Characterization, Mitigation, and Recovery. In *DSN*.
- [10] Yu Cai, Yixin Luo, Erich F. Haratsch, Ken Mai, and Onur Mutlu. 2015. Data Retention in MLC NAND Flash Memory: Characterization, Optimization, and Recovery. In *HPCA*.
- [11] Yu Cai, Onur Mutlu, Erich F. Haratsch, and Ken Mai. 2013. Program Interference in MLC NAND Flash Memory: Characterization, Modeling, and Mitigation. In *ICCD*.
- [12] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F. Haratsch, Adrian Crista, Osman S. Unsal, et al. 2013. Error Analysis and Retention-Aware Management for NAND Flash Memory. *Intel Tech. J.* (2013).
- [13] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F. Haratsch, Adrian Crista, Osman S. Unsal, et al. 2012. Flash Correct-and-Refresh: Retention-Aware Error Management for Increased Flash Memory Lifetime. In *ICCD*.
- [14] Jiho Cho, D Chris Kang, Jongyeol Park, Sang-Wan Nam, Jung-Ho Song, Bong-Kil Jung, Jaedong Lyu, Hogil Lee, Won-Tae Kim, Hongsoo Jeon, et al. 2021. 30.3 A 512Gb 3b/Cell 7th-Generation 3D-NAND Flash Memory with 184MB/s Write Throughput and 2.0 Gb/s Interface. In *ISSCC*.
- [15] Sungjun Cho, Beomjun Kim, Hyunuk Cho, Gyeongseob Seo, Onur Mutlu, Myungsuk Kim, and Jisung Park. 2024. AERO: Adaptive Erase Operation for Improving Lifetime and Performance of Modern NAND Flash-Based SSDs. In *ASPLOS*.
- [16] Wanik Cho, Jongseok Jung, Jongwoo Kim, Junghoon Ham, Sangkyu Lee, Yujong Noh, Dauni Kim, Wanseob Lee, Kayoung Cho, Kwanho Kim, et al. 2022. A 1-Tb, 4b/Cell, 176-Stacked-WL 3D-NAND Flash Memory with Improved Read Latency and a 14.8 Gb/mm² Density. In *ISSCC*.
- [17] Eun-Seok Choi and Sung-Kye Park. 2012. Device Considerations for High Density and Highly Reliable 3D NAND Flash Cell in Near Future. In *IEDM*.
- [18] Myoungjun Chun, Myungsuk Kim, Dusol Lee, Jisung Park, and Jihong Kim. 2024. ReadGuard: Integrated SSD Management for Priority-Aware Read Performance Differentiation. *ACM TOS* (2024).
- [19] Myoungjun Chun, Jaeyong Lee, Myungsuk Kim, Jisung Park, and Jihong Kim. 2024. RiF: Improving Read Performance of Modern SSDs Using an On-Die Early-Retry Engine. In *HPCA*.
- [20] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *SoCC*.
- [21] Graham Cormode and Marios Hadjieleftheriou. 2008. Finding Frequent Items in Data Streams. *PVLDB* (2008).
- [22] Graham Cormode and Marios Hadjieleftheriou. 2010. Methods for Finding Frequent Items in Data Streams. *The VLDB Journal* (2010).
- [23] Jinhua Cui, Zhimin Zeng, Jianhang Huang, Weiqi Yuan, and Laurence T Yang. 2022. Improving 3D NAND SSD Read Performance by Parallelizing Read-Retry. *IEEE TCAD* (2022).
- [24] Richard G Forbes. 2008. On the Need for a Tunneling Pre-Factor in Fowler–Nordheim Tunneling Theory. *Journal of Applied Physics* (2008).
- [25] Keonsoo Ha, Jaeyong Jeong, and Jihong Kim. 2015. An Integrated Approach for Managing Read Disturbs in High-Density NAND Flash Memory. *IEEE TCAD* (2015).
- [26] Sangwoo Han, Minjung Cho, Gi Lee, and Eui-Young Chung. 2023. Page Type-Aware Data Migration Technique for Read Disturb Management of NAND Flash Memory. *IEEE TVLSI* (2023).
- [27] Mingzhe Hao, Gokul Soundararajan, Deepak Kenchammana-Hosekote, Andrew A Chien, and Haryadi S Gunawi. 2016. The Tail at Store: A Revelation from Millions of Hours of Disk and SSD Deployments. In

FAST.

- [28] Duwon Hong, Myungsuk Kim, Geonhee Cho, Dusol Lee, and Jihong Kim. 2022. GuardedErase: Extending SSD Lifetimes by Protecting Weak Wordlines. In *FAST*.
- [29] SK Hynix. 2023. SK Hynix Enterprise SSDs. <https://product.skhynix.com/products/ssd/essd.go>.
- [30] Jaehoon Jang, Han-Soo Kim, Wonseok Cho, Hoosung Cho, Jinho Kim, Sun Il Shim, Jae-Hun Jeong, Byoung-Keun Son, Dong Woo Kim, Jae-Joo Shim, et al. 2009. Vertical Cell Array Using TCAT (Terabit Cell Array Transistor) Technology for Ultra High Density NAND Flash Memory. In *VLSI*.
- [31] JEDEC. 2018. JESD22-A117: Electrically Erasable Programmable ROM (EEPROM) Program / Erase Endurance and Data Retention Stress Tests. <https://www.jedec.org/sites/default/files/docs/22A117B.pdf>
- [32] JEDEC. 2022. JESD47L: Stress-Test-Driven Qualification of Integrated Circuits. <https://www.jedec.org/standards-documents/docs/jesd-47g>
- [33] Ali Khakifirooz, Sriram Balasubrahmanyam, Richard Fastow, Kristopher H Gaewsky, Chang Wan Ha, Rezaul Haque, Owen W Jungroth, Steven Law, Aliasgar S Madraswala, Binh Ngo, et al. 2021. 30.2 A 1Tb 4b/Cell 144-Tier Floating-Gate 3D-NAND Flash Memory with 40MB/s Program Throughput and 13.8Gb/mm² Bit Density. In *ISSCC*.
- [34] Bvunarvul Kim, Seungpil Lee, Beomseok Hah, Kanawoo Park, Yongsoon Park, Kangwook Jo, Yujong Noh, Hyeoncheon Seol, Hyunsoo Lee, Jaehyeon Shin, et al. 2023. 28.2 A High-Performance 1Tb 3b/Cell 3D-NAND Flash with a 194MB/s Write Throughput on over 300 Layers i. In *ISSCC*.
- [35] Bryan S Kim, Jongmoo Choi, and Sang Lyul Min. 2019. Design Trade-offs for SSD Reliability. In *FAST*.
- [36] Earl Kim, Hyunuk Cho, Sungjun Cho, Myungsuk Kim, Jisung Park, Jaeyong Jeong, Eunkyong Kim, and Sunghoi Hur. 2024. NORN: Three Guides for Efficient Automatic Post-Fabrication Optimization of Modern NAND Flash Memory. In *JCCAD*.
- [37] Myungsuk Kim, Myoungjun Chun, Duwon Hong, Yoona Kim, Geonhee Cho, Dusol Lee, and Jihong Kim. 2021. RealWear: Improving Performance and Lifetime of SSDs Using a NAND Aging Marker. *Performance Evaluation* (2021).
- [38] Myungsuk Kim, Jaehoon Lee, Sungjin Lee, Jisung Park, Youngsun Song, and Jihong Kim. 2017. Improving Performance and Lifetime of Large-Page NAND Storages Using Erase-Free Subpage Programming. In *DAC*.
- [39] Shine Kim, Jonghyun Bae, Hakbeom Jang, Wenjing Jin, Jeonghun Gong, Seungyeon Lee, Tae Jun Ham, and Jae W Lee. 2019. Practical Erase Suspension for Modern Low-Latency SSDs. In *USENIX ATC*.
- [40] Sang-Hoon Kim, Jaehoon Shim, Euidong Lee, Seongyeop Jeong, Ilkueon Kang, and Jin-Soo Kim. 2023. NVMeVirt: A Versatile Software-Defined Virtual NVMe Device. In *FAST*.
- [41] Dusol Lee, Duwon Hong, Wonil Choi, and Jihong Kim. 2022. MQSim-E: An Enterprise SSD Simulator. *IEEE CAL* (2022).
- [42] Seungjae Lee, Jin-yub Lee, Il-han Park, Jongyeol Park, Sung-won Yun, Min-su Kim, Jong-hoon Lee, Minseok Kim, Kangbin Lee, Taeun Kim, et al. 2016. 7.5 A 128Gb 2b/Cell NAND Flash Memory in 14nm Technology with tPROG=640μs and 800MB/s I/O Rate. In *ISSCC*.
- [43] Jun Li, Bowen Huang, Zhibing Sha, Zhigang Cai, Jianwei Liao, Balazs Gerofi, and Yutaka Ishikawa. 2020. Mitigating Negative Impacts of Read Disturb in SSDs. *TODAES* (2020).
- [44] Qiao Li, Min Ye, Yufei Cui, Liang Shi, Xiaoqiang Li, Tei-Wei Kuo, and Chun Jason Xue. 2020. Shaving Retries with Sentinels for Fast Read over High-Density 3D Flash. In *MICRO*.
- [45] Jianwei Liao, Jun Li, Mingwang Zhao, Zhibing Sha, and Zhigang Cai. 2022. Read Refresh Scheduling and Data Reallocation against Read Disturb in SSDs. *ACM TECS* (2022).
- [46] Heiner Litz, Javier Gonzalez, Ana Klimovic, and Christos Kozyrakis. 2022. RAIL: Predictable, Low Tail Latency for NVMe Flash. *ACM TOS* (2022).
- [47] Yixin Luo, Saugata Ghose, Yu Cai, Erich F. Haratsch, and Onur Mutlu. 2018. Improving 3D NAND Flash Memory Lifetime by Tolerating Early Retention Loss and Process Variation. In *SIGMETRICS*.
- [48] Richard McDougall and Jim Mauro. 2005. FileBench. <https://github.com/filebench/filebench>.
- [49] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2005. Efficient Computation of Frequent and Top-k Elements in Data Streams. In *ICDT*.
- [50] Rino Micheloni, Luca Crippa, and Alessia Marelli. 2010. *Inside NAND Flash Memories*. Springer Science & Business Media.
- [51] Micron. 2006. NAND Flash Design and Use Considerations. <https://media-www.micron.com/-/media/client/global/documents/products/technical-note/nand-flash/tn2917.pdf>
- [52] Micron. 2019. SSDs and SMART Data. <https://www.crucial.com/support/articles-faq-ssd/ssds-and-smart-data>.
- [53] Micron. 2023. 232-Layer NAND Flash Memory. <https://www.micron.com/products/nand-flash/232-layer-nand>.
- [54] ONFI Workgroup. 2024. Open NAND Flash Interface Specification Revision 5.2. https://onfi.org/files/ONFI_5_2_Rev1.0.pdf
- [55] Yuqian Pan, Haichun Zhang, Mingyang Gong, and Zhenglin Liu. 2020. Process-Variation Effects on 3D TLC Flash Reliability: Characterization and Mitigation Scheme. In *QRS*.
- [56] Jisung Park, Roknoddin Azizi, Geraldo F Oliveira, Mohammad Sadrosadati, Rakesh Nadig, David Novo, Juan Gómez-Luna, Myungsuk Kim, and Onur Mutlu. 2022. Flash-Cosmos: In-Flash Bulk Bitwise Operations Using Inherent Computation Capability of NAND Flash Memory. In *MICRO*.
- [57] Jisung Park, Jaeyong Jeong, Sungjin Lee, Youngsun Song, and Jihong Kim. 2016. Improving Performance and Lifetime of NAND Storage Systems Using Relaxed Program Sequence. In *DAC*.
- [58] Jisung Park, Myungsuk Kim, Myoungjun Chun, Lois Orosa, Jihong Kim, and Onur Mutlu. 2021. Reducing Solid-State Drive Read Latency by Optimizing Read-Retry. In *ASPLOS*.
- [59] Jae-Woo Park, Doogon Kim, Sunghwa Ok, Jaebeom Park, Taeheui Kwon, Hyunsoo Lee, Sungmook Lim, Sun-Young Jung, Hyeonjin Choi, Taikyung Kang, et al. 2021. 30.1 A 176-Stacked 512Gb 3b/Cell 3D-NAND Flash with 10.8 Gb/mm² Density with a Peripheral Circuit Under Cell Array Architecture. In *ISSCC*.
- [60] Suk Kwang Park and Jaekyun Moon. 2021. Characterization of Inter-Cell Interference in 3D NAND Flash Memory. *IEEE TCAS-I* (2021).
- [61] Tianyu Ren, Qiao Li, Min Ye, and Chun Jason Xue. 2023. Read Disturb and Reliability: The Complete Story for 3D CT NAND Flash. In *NVMSA*.
- [62] Samsung. 2015. Communicating With Your SSD: Understanding SMART Attributes. https://web.archive.org/web/20150402093935/http://www.samsung.com/global/business/semiconductor/minisite/SSD/downloads/document/07_Communicating_With_Your_SSD.pdf.
- [63] Samsung. 2019. QLC SSD: High Density, High Performance and Affordable Solution for Client PCs. https://download.semiconductor.samsung.com/resources/white-paper/Samsung_SSD_860_QVO_White_paper.pdf.
- [64] Samsung. 2023. Samsung Enterprise SSDs. <https://semiconductor.samsung.com/ssd/enterprise-ssd/>.
- [65] Nikhil Seshasayee and Keith Neil MacLean. 2018. Cascaded E-Fuse Switch Circuits to Control Data Backup in a Storage Device. US Patent 9,984,762.
- [66] Liang Shi, Yejia Di, Mengying Zhao, Chun Jason Xue, Kaijie Wu, and Edwin H-M Sha. 2015. Exploiting Process Variation for Write Performance Improvement on NAND Flash Memory Storage Systems. *IEEE TVLSI* (2015).
- [67] Youngseop Shim, Myungsuk Kim, Myoungjun Chun, Jisung Park, Yoona Kim, and Jihong Kim. 2019. Exploiting Process Similarity of 3D Flash Memory for High Performance SSDs. In *MICRO*.

- [68] Ikjoon Son and Jin-Soo Kim. 2023. Efficient Read Disturb Management Schemes in Resource-Constrained Flash Memory Controller. In *NVMSA*.
- [69] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. 2018. MQSim: A Framework for Enabling Realistic Studies of Modern Multi-Queue SSD Devices. In *FAST*.
- [70] Arash Tavakkol, Mohammad Sadrosadati, Saugata Ghose, Jeremie Kim, Yixin Luo, Yaohua Wang, Nika Mansouri Ghiasi, Lois Orosa, Juan Gómez-Luna, and Onur Mutlu. 2018. FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives. In *ISCA*.
- [71] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and Efficient Foundation Language Models. *arXiv* (2023).
- [72] Keval Vora. 2019. LUMOS: Dependency-Driven Disk-Based Graph Processing. In *USENIX ATC*.
- [73] Yi Wang, Jiangfan Huang, Jing Chen, and Rui Mao. 2021. PVSensing: a Process-Variation-Aware Space Allocation Strategy for 3D NAND Flash Memory. *IEEE TCAD* (2021).
- [74] Guanying Wu and Xubin He. 2012. Reducing SSD Read Latency via NAND Flash Program and Erase Suspension. In *FAST*.
- [75] Qin Xiong, Fei Wu, Zhonghai Lu, Yue Zhu, You Zhou, Yibing Chu, Changsheng Xie, and Ping Huang. 2018. Characterizing 3D Floating Gate NAND Flash: Observations, Analyses, and Implications. *ACM TOS* (2018).
- [76] Shiqin Yan, Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminathan Sundararaman, Andrew A Chien, and Haryadi S Gunawi. 2017. Tiny-Tail Flash: Near-Perfect Elimination of Garbage Collection Tail Latencies in NAND SSDs. *ACM TOS* (2017).
- [77] Min Ye, Qiao Li, Yina Lv, Jie Zhang, Tianyu Ren, Daniel Wen, Tei-Wei Kuo, and Chun Jason Xue. 2024. Achieving Near-Zero Read Retry for 3D NAND Flash Memory. In *ASPLOS*.
- [78] Kong-Kiat Yong and Li-Pin Chang. 2020. Error Diluting: Exploiting 3-D NAND Flash Process Variation for Efficient Read on LDPC-Based SSDs. *IEEE TCAD* (2020).
- [79] Kong-Kiat Yong and Li-Pin Chang. 2020. Error Diluting: Exploiting 3D NAND Flash Process Variation for Efficient Read on LDPC-Based SSDs. *IEEE TCAD* (2020).
- [80] Cristian Zambelli, Piero Olivo, Luca Crippa, Alessia Marelli, and Rino Micheloni. 2017. Uniform and Concentrated Read Disturb Effects in Mid-1X TLC NAND Flash Memories for Enterprise Solid State Drives. In *IRPS*.
- [81] Genxiong Zhang, Yuhui Deng, Yi Zhou, Shujie Pang, Jianhui Yue, and Yifeng Zhu. 2022. Cocktail: Mixing Data with Different Characteristics to Reduce Read Reclaims for NAND Flash Memory. *IEEE TCAD* (2022).
- [82] Jie Zhang, Miryeong Kwon, Donghyun Gouk, Sungjoon Koh, Changlim Lee, Mohammad Alian, Myoungjun Chun, Mahmut Taylan Kandemir, Nam Sung Kim, Jihong Kim, et al. 2018. FlashShare: Punching Through Server Storage Stack from Kernel to Firmware for Ultra-Low Latency SSDs. In *OSDI*.
- [83] Mingwang Zhao, Jun Li, Zhigang Cai, Jianwei Liao, and Yuanquan Shi. 2021. Block Attribute-Aware Data Reallocation to Alleviate Read Disturb in SSDs. In *DATE*.
- [84] Timothy Zhu, Michael A Kozuch, and Mor Harchol-Balter. 2017. WorkloadCompactor: Reducing Datacenter Cost While Providing Tail Latency SLO Guarantees. In *SoCC*.